

A language is a set of strings. Strings are sequences over  $\Sigma$ .  
A language over  $\Sigma$ : A set of strings whose characters are from the alphabet  $\Sigma$ . A grammar: a set of rewrite rules that (recursively) generate strings in a language.

### Syntax of REs

**Definition:** Let  $\Sigma$  be an alphabet. The regular expressions over  $\Sigma$  are defined by the following recursive process (Rules):

- The symbols  $\emptyset$  and  $\epsilon$  are regular expressions
- Each symbol  $a$  from  $\Sigma$  is a regular expression
- If  $R_1, R_2$  are regular expressions then so is  $(R_1 \mid R_2)$
- If  $R_1, R_2$  are regular expressions then so is  $(R_1 R_2)$
- If  $R$  is a regular expressions then so is  $R^*$

### Shorthands

**Author: Wenyu (Wendy) Fan**

- $(a \mid \emptyset) \rightarrow a \mid \emptyset$
- $(ab^*) \rightarrow ab^*$
- $((R_1 \mid R_2) \mid R_3) \rightarrow (R_1 \mid R_2 \mid R_3)$
- $((R_1 R_2) R_3) \rightarrow (R_1 R_2 R_3)$
- if  $A = \{a, b, c\}$ ,  $(a \mid b \mid c) \rightarrow A$ .
- $AA \rightarrow A^2$
- $\emptyset =$  empty set, does not match any string. same with  $\emptyset \mid \emptyset$ .
- $\emptyset$  has zero strings.  $\{\epsilon\}$  has one string (the string of length 0).
- $\epsilon R = R\epsilon = R$  and  $\emptyset R = R\emptyset = \emptyset$
- $\epsilon^* = \{\epsilon\}$ , and  $\emptyset^* = \{\epsilon\}$ , but  $\emptyset^+ = \emptyset$

### Operation on languages (used to reason abt REs)

**Concatenation:**  $AB = \{xy \mid x \in A, y \in B\}$

- $A^0 = \{\epsilon\}$
- $A^{n+m} = A^n A^m$  (even if  $n$  or  $m$  is zero).
- $\{00, 11\}^2 = \{0000, 0011, 1100, 1111\}$
- Kleene star:**  $A^* = A^0 \cup A^1 \cup A^2 \cup A^3 \cup \dots$
- $A^+ = A^1 \cup A^2 \cup A^3 \cup \dots$

### Semantics of REs (Language represented by a RE) - Definition

We define **Lang(R)** recursively as follows:

- $\text{Lang}(\epsilon) = \{\epsilon\}$
- $\text{Lang}(\emptyset) = \{\}$
- $\text{Lang}(a) = \{a\}$  (for  $a \in \Sigma$ )
- $\text{Lang}(R_1 \mid R_2) = \text{Lang}(R_1) \cup \text{Lang}(R_2)$
- $\text{Lang}(R_1 R_2) = \text{Lang}(R_1) \text{Lang}(R_2)$
- $\text{Lang}(R^*) = \text{Lang}(R)^*$

A string  $s$  matches  $R$  if  $s$  is in the set  $\text{Lang}(R)$ .

### Automata $\rightarrow$ decide if a RE $R$ matches a string $w$ .

**DFA - Definition:**  $(Q, \Sigma, \delta, q_0, F)$ ; doesn't have  $\epsilon$ -transitions

- States  $(Q)$  - finite set of states.
- Alphabet  $(\Sigma)$  - allowed input symbols.
- Transition function  $(\delta)$ :  $Q \times \Sigma \rightarrow Q$ 
  - Transition:  $\delta(q, a) = q' \equiv q \xrightarrow{a} q'$
- Start state  $(q_0 \in Q, q_0)$  - initial state.
- Final states  $(F \subseteq Q)$  - accepting states.

**Lang(D)** = all accepted strings (no more, no less).

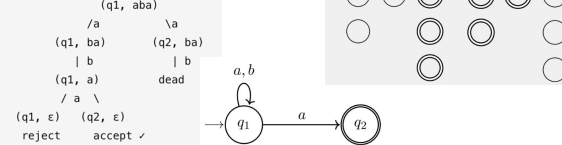
### Boolean operations on DFA:

- Complement  $(\sim L_1)$ :** When you have a DFA (not NFA): swap final and non-final states.
  - Union  $(L_1 \cup L_2)$**  - product construction
  - Intersection  $(L_1 \cap L_2)$**  - product construction
  - Concatenation  $(L_1 L_2)$**  - do it via NFA, then subset construction to DFA
  - Kleene star  $(L_1)^*$**  - also via NFA
- NFA - Definition:** can return multiple next states (or none)
- Accepting path:** a string is accepted by an NFA if it labels some path from the start state to a final stet.
  - "nondeterminism"** refers to situations in which the next state of a computation is **not uniquely determined** by the current state and current input.

### Comparison

- DFA:** exactly one run per input; accepted if this run is accepting.
- NFA:** may have 0/1/many runs; accepted if  $\geq 1$  is accepting.

### Computation Tree

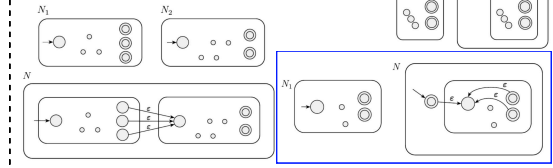


### 1. REs $\rightarrow$ NFAs

- idea: build NFA for the union/cat/star of NFAs
- NFA for the union of NFAs:**

- if applied to 2 DFAs,
- result still be NFA ( $\epsilon!$ )
- RE  $ab^* \rightarrow$  NFA:  $2+2=4$  states

### NFA for the cat of NFAs; NFA for the star

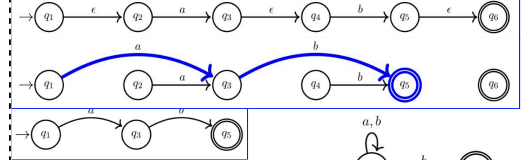


### 2. NFAs $\rightarrow$ NFAs without $\epsilon$ -transitions

- idea: skip  $\epsilon$ -transitions
  - does not create any new states**
- Notation: Write  $\text{EC}(q, r)$  (stand for "Epsilon Closure") or  $q \rightsquigarrow^* r$  to mean that  $N$  can go from  $q$  to  $r$  using zero or more  $\epsilon$ -transitions.

Steps (for all states  $q, r, s$  of  $N$ , and all characters  $a \in \Sigma$ ):

- If  $q \rightsquigarrow^* r$  and  $r \xrightarrow{a} s$  then add  $q \xrightarrow{a} s$  to  $M$ .
- If  $q \rightsquigarrow^* r$  and  $r$  is final, make  $q$  final in  $M$ .



**morphett notation rule:** <current state> <current cell> <new cell>  
**TMs Properties:**

- input alphabet  $\Sigma$ : set of symbols allowed to take as input; no blank ( $\_$ ) accepted.
- tape alphabet  $\Gamma$ : full set of symbols for read and write during computation. include blank and work markers ( $\#x,y,z$ )
- A TM does not have to read the whole input to accept.
- If a TM only move right, it is like a DFA.
- If a TM M does not halt/accept any inputs,  $\text{Lang}(M)=\text{the empty set } (= \emptyset = \{\})$

**Lang(M)** = set of strings M accepts.  
 $L_{\text{empty}} = \{\text{Source\_M} \mid \text{Lang}(M) = \emptyset\}$  is not decidable and not recog.

**1. Python  $\rightleftharpoons$  TM (Theorem) (same input/output behaviour):**  
**Recall:** two programming languages or models of computation are **expressively equivalent** if they can solve the same computational problems.

**Goal:** show TMs are expressively equivalent to Python.

**Two directions:**

- Python can simulate any TM (e.g., Morphet simulators).
- A TM can simulate any Python program by first restricting to Tiny Python (inc/dec/while) and implementing each in a TM, then composing (sequencing, nesting).

**Conclusion:** Python  $\rightleftharpoons$  TM (same input/output behaviour).  
**2. Church–Turing Thesis:** for every algorithm, its function from inputs to outputs can be recognised by a TM.

**Recall:** The input/output behaviour of an algorithm, Python Program, or TM, refers to what happens to every valid input. I.e., whether or not it halts on that input, and if it does, what the corresponding output is.

**CTT: for every algorithm, its function from inputs to outputs can be recognised by a TM.**

**Goal:** show If a Python Program implements an algorithm, there is a TM that implements the same algorithm (and vice versa).

**Why people accept CTT:**

- Algorithms are at least as expressive as Turing Machines: All known models of general computation are expressively equivalent to TMs.
- Algorithms are no more expressive as Turing Machines: Problems beyond TMs don't appear to be algorithmic.

3. Together, we accept CTT that: Algorithms and TMs are expressively equivalent (algorithm  $\approx$  TM)

**Applications:**  
For each model of computation, we can pose the **acceptance problem** (aka membership problem).

**DFA acceptance problem:** A Python Program with

- Input: D (DFA) and w (string).
- Output: decide whether D accepts w.

- This algorithm always halts and is correct (decider).
- Encode inputs (and outputs) as strings.
- Apply CTT $\Rightarrow$  there exists a TM with the same yes/no behaviour  $\Rightarrow$  problem is Turing-decidable

**CFG membership problem:**

- Input: G (CFG) and w (string)
- Output: decide whether  $w \in \text{Lang}(G)$ .

- Have an algorithm
  - Convert G to CNF
  - run CYK on CNF(G) and w
  - this halts and is correct
- Encode inputs (and outputs) as strings
- Apply CTT $\Rightarrow$  there exists a TM with the same yes/no behaviour  $\Rightarrow$  problem is Turing-decidable

**How to prove recognisable languages are closed under union.**

- Show there is a TM M that recognises  $\text{Lang}(M1) \cup \text{Lang}(M2)$ .
- By the CTT, M works as follows on input x: it runs M1 on x "in parallel" with M2 on x (this can be done with a two-tape machine: the first tape simulates M1 and the second tape simulates M2).
- If at any step M1 accepts, then M halts and accepts; and if at any step M2 accepts, then M halts and accepts.

<direction> <new state>  
**3. Universal Turing Machines (UTMs)**

- Input:** the **source code** of a TM M and an input string s for M, i.e., an encoding of  $(\text{Source\_M}; s)$
- Simulates M running on s:** halts/accepts/rejects exactly when M does
- Output:** if M has an output on s, U has the same output, but may run in more steps than M).
- UTM != decider** (it doesn't halt when M doesn't).
- Multitape TM  $\equiv$  Single-tape TM:** simulate multiple tapes on one tape using separators.
- Lang(U):**  $L_{\text{TM-accept}} = \{\text{Source\_M}; s \mid M \text{ accepts } s\}$ .

**Universality via the CTT**

- The Morphet Simulator is a program X that takes as input a string encoding a TM M, and an input string s, and returns as output: the output tape of M running on s— with identical halting behaviour.
- By the CTT, there is a TM U that simulates X.
- U by definition is a UTM.

**Binary alphabet simulation:**  
Ordinary TMs (i.e., with any finite alphabet) are expressively equivalent to Binary TMs (tape alphabet is just  $\{0,1,\_ \}$ ).

**Applications:**

- Bootstrapping:** languages interpret/compile themselves
- The Limits of Computation**
- Recognisable:**

- some TM accepts exactly the language (may loop on non-members).
- closed under  $\cup, \cap$ , concat, star
- not closed under complement

**Decidable:**

- recognised by a TM that halts on all inputs.
- closed under  $\cup, \cap$ , complement, concat, star.

**Theorem:**

- if L and  $\neg L$  are both recognizable, then L is decidable.
- if L is undecidable,  $\neg L$  is recognizable, then L is unrecog-.

**Undecidability – problems that TMs cannot solve.**

- L\_TM-not-accept = {Source\_M; w | M is a TM that does not accept w} is not recognisable.**  
(diagonal proof – not examinable).
- L\_TM-accept = {<M; w> | M is a TM that accepts w} is undecidable. (recogniser)**
  - This shows that Turing-decidable != Turing-recognisable.
  - derive untrue statement: The  $L_{\text{diag}}$  is recognisable, then contradiction.

**The Halting Problem:**

- L\_TM-halt = {Source\_M; w | M is a TM that halts on w} is undecidable.**

**Proof structure:**  
**Goal:** use this to build a decider for TM-acceptance problem, which is undecidable.

- Assume decider H for L\_TM-halt:**  $H(\text{sourceM}; w)$  accepts if M halts on w, and rejects otherwise.
- Build decider D for L\_TM-accept.** def  $D(M, w)$ .
  - On input (M; w), Run  $H(M; w)$ ): Two cases:**
    - if not  $H(M, w)$ : return "reject"
    - else: simulate M on w until it halts and accept/reject as M does.
    - Then D decides L\_TM-accept (contradiction)

- L\_TM-halt = {Source\_M; w | M is a TM that halts on w} is recognisable. (recogniser)**

**Proof structure:**  
**Goal:** find a TM H that recognises  $L_{\text{TM-halt}} \rightarrow \text{UTM}$ .

- def  $\text{pyH}(x)$ :  $\text{pyU}(x)$  return True
- Two cases:
  - if H accepts x,  $\Rightarrow$  U accepts x  $\Rightarrow$  x is of the form  $\text{Source\_M}; w$  (for some M) and M halts on w (otherwise U wouldn't have).
  - if H does not accept x  $\Rightarrow$  U does not return on input x  $\Rightarrow$  if x is of the form  $\text{Source\_M}; w$ , we know that M does not halt on w.
- So,  $\text{Lang}(H) = L_{\text{TM-halt}}$ .

**Predicate logic (aka First-order logic)**  
**Syntax**

- domain:** a non-empty set D, collection of objects,
- predicates:** properties of objects and relations between objects.
  - A predicate (of arity k) over domain  $\mathbb{D}$  is a subset of  $\mathbb{D}^k$ .
    - happy  $\subseteq \mathbb{H}$  is a unary predicate ( $k = 1$ ).
    - loves  $\subseteq \mathbb{H} \times \mathbb{H}$  is a binary predicate ( $k = 2$ ).
  - Infix notation: x loves y
  - Functional notation: loves:  $\mathbb{H} \times \mathbb{H} \rightarrow \{\text{true}, \text{false}\}$
- variables:** vary over objects in the domain.
- quantifiers:** allow one to reason about multiple objects.

**Transition Tips**

- $\wedge$  goes with  $\exists$ ;  $\rightarrow$  goes with  $\forall$

**Common forms:**

- "All As are Bs":  $\forall x(A(x) \rightarrow B(x))$
- "Some As are Bs":  $\exists x(A(x) \wedge B(x))$
- "No As are Bs":  $\forall x(A(x) \rightarrow \neg B(x))$
- "Some As are not Bs":  $\exists x(A(x) \wedge \neg B(x))$

**The order of quantifiers only matters when mixing  $\exists$  and  $\forall$ :**

- $\exists x \forall y P(x) = \forall x P(x)$ : inner  $\exists/\forall$  shadows outer  $\exists/\forall$
- $\forall x \exists y \text{ loves}(x, y)$ : everyone loves someone.
- $\exists y \forall x \text{ loves}(x, y)$ : there is one person loved by everyone.

**Bound/Free variables**

- bound:** x in G is bound if there is a  $\forall x$  or  $\exists x$  that wraps G(x). e.g.,  $\forall x(\exists y(G(x, y)))$
- free:** otherwise
- sentence:** a formula without free variables

**Decidable** = Recognisable  $\cap$  co-Recognisable.  
**R but not D:**  $L_{\text{TM-accept}}, L_{\text{TM-halt}}$ . (complements not R).  
**Not R but co-R:** complement of an R-but-not-D language (e.g.  $L_{\text{TM-not-accept}}, L_{\text{TM-not-halt}}$ ).  
**Neither R nor co-R:** e.g.  $L_{\text{diag}}$ .

**Common TM:**

- $L_{\text{TM-not-halt}}$ : not recognisable
- $L_{\text{invalid}} = \{\text{strings that aren't valid } \langle M \rangle; w \text{ encodings}\}$ : decidable.

**Syntax properties  $\rightarrow$  decidable:**

- TM depend only on the written description/encoding of the TM  $\rightarrow$  **decidable**.
- e.g., " $\geq 10$  states", "uses alphabet  $\{0,1\}$  only", "never moves left", " $\leq 50$  rules".

**Semantic properties  $\rightarrow$  undecidable:**

- TM depend on what language the TM computes or how it runs  $\rightarrow$  **undecidable**.
- e.g., "halts on empty input", "accepts all strings", "its lang is finite", "ever prints a 1"

undecidable languages are also closed under complement  
**Prove that if  $L = \{\text{SourceM} : M \text{ is a TM that accepts the string aab}\}$  were decidable, then one could decide the following language:  $A = \{\text{SourceM} : M \text{ is a TM that accepts } \epsilon\}$**

Let  $D_L$  be a decider for L.  
Build a decider  $D_A$  for A. On input  $\langle M \rangle$ :

- build N: "ignore input; run M on  $\epsilon$ "
- runs  $D_L(N)$  and returns what  $D_L$  returns.

Why this construction is correct:

- $D_L$  is a decider  $\rightarrow D_A$  is a decider.
- $D_A$  accepts  $\langle M \rangle$ 
  - $\Leftrightarrow D_L$  accepts  $\langle N \rangle$  (by definition of  $D_A$ )
  - $\Leftrightarrow N$  accepts "aab" (because  $D_L$  decides L)
  - $\Leftrightarrow M$  accepts  $\epsilon$ .
- If M accepts  $\epsilon$ , then N (ignoring its input and running M on  $\epsilon$ ) accepts every input  $\Rightarrow$  in particular accepts "aab".
- If N accepts "aab", then by construction N ran M on  $\epsilon$  and M accepted.

**Proposition Logic**

- A **proposition** is a sentence that declares a fact that is either true, or false, but not both.

**Syntax**

- atom:** a variable of the form p, q, r...
- formula:** defined by the following recursive process:
  - every atom is a formula
  - if F is a formula then  $\neg F$  is a formula
  - if F, G are formulas then so are  $(F \vee G)$  and  $(F \wedge G)$ .
- no  $()$  in atoms:  $(\neg p)$  is not a formula
- T: top/verum
- $\perp$ : bottom/falsum

**Semantics**  
**Truth values**

- assignment:** a function  $\alpha$  from atoms to truth values.
- truth value**  $\text{tv}(F, \alpha) = \alpha(F)$ : a formula F under assignment  $\alpha$
- In case  $\alpha(F) = 1$ , we say  $\alpha \models F$  ( $\alpha$  satisfies F).
- $E \models F$  means exactly the same thing as " $(E \rightarrow F)$  is valid".

**Logical symbols**

- T and  $\perp$  are formulas; called the **propositional constants**.
- if F, G are formulas then so are  $(F \rightarrow G)$  and  $(F \leftrightarrow G)$ .

**Conditional:** if F is not true, the formula  $(F \rightarrow G)$  is true.  
**Validity:** every assignment satisfies F.  
**Satisfiability:** at least one assignment satisfies F.

**1. Logical equivalence**

- logical formula:  $E \leftrightarrow F$ .
- mathematical statement about logical formulas:  $E \equiv F$ .

**2. Normal forms**

- Theorem:** For every formula F there is equivalent formula in NNF/CNF.

**2.1 Negation normal form (NNF):**

- every  $\neg$  must be directly in front of an atom, not in front of a bigger formula.

**2.2 Conjunctive normal form (CNF):**

- if it is a **clause** or a **conjunction of clauses** ( $\wedge$  of  $\vee$ -clauses).
- literal:** p or  $\neg p$  ( $\neg \neg p$  is not a literal)
- clause:** a literal or a  $\vee$  of literals.
  - $(p \vee \neg q \vee r)$  and  $\neg r$  are clauses;  $(p \vee q) \wedge r$  isn't.

**2.3 Disjunctive normal form (DNF): ( $\vee$  of  $\wedge$ -terms)**

**3. Logical consequence**

- F is a **logical consequence** of A, B, ..., D if every assignment that make all formulas A, B, ..., D true also makes F true.
- write as: A, B, ...,  $D \models F$
- If no LHS, then  $\models F$  means F is valid.

**Implication elimination / Modus Ponens:**

1.  $p \rightarrow q$ ; 2. p; 3. Conclude, q.

**Reasoning by cases / disjunction elimination:**

1.  $h \rightarrow t$ ; 2.  $w \rightarrow t$ ; 3.  $h \vee w$ ; 4. Conclude t.

**The error of the converse:**

1.  $p \rightarrow q$ ; 2. q; 3. Conclude, p.

$\cdot \forall v(p \wedge q) \Rightarrow q$

Idempotent:  $F \equiv (F \wedge F)$   
 $F \equiv (F \vee F)$

Commutative:  $(F \wedge G) \equiv (G \wedge F)$   
 $(F \vee G) \equiv (G \vee F)$

Associative:  $(F \wedge (G \wedge H)) \equiv ((F \wedge G) \wedge H)$   
 $(F \vee (G \vee H)) \equiv ((F \vee G) \vee H)$

Absorption:  $(F \wedge (G \vee G)) \equiv F$   
 $(F \vee (F \wedge G)) \equiv F$

Distributive:  $(F \wedge (G \vee H)) \equiv ((F \wedge G) \vee (F \wedge H))$   
 $(F \vee (G \wedge H)) \equiv ((F \vee G) \wedge (F \vee H))$

de Morgan:  $\neg(F \wedge G) \equiv (\neg F \vee \neg G)$   
 $\neg(F \vee G) \equiv (\neg F \wedge \neg G)$

Double Negation:  $\neg \neg F \equiv F$

Validity:  $(F \vee T) \equiv T$   
 $(F \wedge T) \equiv F$

Unsatisfiability:  $(F \vee \perp) \equiv F$   
 $(F \wedge \perp) \equiv \perp$

Constant:  $T \equiv (F \vee \neg F)$   
 $\perp \equiv (F \wedge \neg F)$

Negating const.:  $\neg T \equiv \perp$   
 $\neg \perp \equiv T$

Conditional:  $(F \rightarrow G) \equiv (\neg F \vee G)$

Bi-conditional:  $(F \leftrightarrow G) \equiv ((F \rightarrow G) \wedge (G \rightarrow F))$  (have same value)

• Every prop formula is logically equivalent to a formula which only contains the connective

- $p \uparrow q \equiv \neg(p \wedge q)$ .
- $p \uparrow p \equiv \neg p$ .
- $(p \uparrow q) \uparrow (p \uparrow q) \equiv p \wedge q$ .